

Practical 1: Substitution and Transposition Ciphers

AIM:

Design and implement algorithms to encrypt and decrypt messages using classical substitution and transposition techniques.

Code:

```
def caesar_encrypt(text, shift):
    result = ""
    for ch in text:
        if ch.isalpha():
            base = 65 if ch.isupper() else 97
            result += chr((ord(ch) - base + shift) % 26 + base)
        else:
            result += ch
    return result

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)

def transposition_encrypt(text, key):
    text = text.replace(" ", "")
    cols = len(key)
    order = sorted(range(cols), key=lambda i: key[i])
    rows = [text[i:i+cols] for i in range(0, len(text), cols)]
    result = ""
    for col in order:
        for row in rows:
            if col < len(row):
                result += row[col]
    return result

msg = "ATTACK AT DAWN"
print("Original:", msg)

enc = caesar_encrypt(msg, 3)
print("Caesar Encrypted:", enc)
print("Caesar Decrypted:", caesar_decrypt(enc, 3))

enc2 = transposition_encrypt(msg, "KEY")
print("Transposition Encrypted:", enc2)
```

Output:

```
PS C:\VS CODE> python -u "c:\VS CODE\Python\CIS\Transposition.py"
Original: ATTACK
Caesar Encrypted: DWWDFN
Caesar Decrypted: ATTACK
Transposition Encrypted: TCAATK
PS C:\VS CODE>
```

Practical 2: RSA Encryption and Decryption

AIM:

Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

Code:

```
from math import gcd

def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d * e) % phi == 1:
            return d
    return None

p, q = 61, 53
n = p * q
phi = (p - 1) * (q - 1)

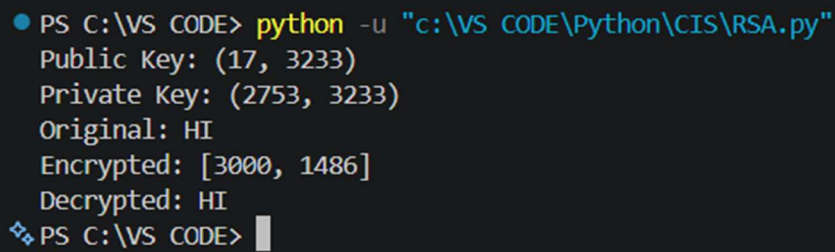
e = 17
while gcd(e, phi) != 1:
    e += 1
d = mod_inverse(e, phi)

print("Public Key:", (e, n))
print("Private Key:", (d, n))

msg = "HI"
encrypted = [pow(ord(ch), e, n) for ch in msg]
decrypted = "".join(chr(pow(c, d, n)) for c in encrypted)

print("Original:", msg)
print("Encrypted:", encrypted)
print("Decrypted:", decrypted)
```

Output:



```
PS C:\VS CODE> python -u "c:\VS CODE\Python\CIS\RSA.py"
Public Key: (17, 3233)
Private Key: (2753, 3233)
Original: HI
Encrypted: [3000, 1486]
Decrypted: HI
PS C:\VS CODE> 
```

Practical 3: Message Authentication Codes (MAC)

AIM:

Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity.

Code:

```
import hmac
import hashlib

key = b"secretkey"
message = b"Transfer $500"

mac = hmac.new(key, message, hashlib.sha256).hexdigest()
print("Message:", message.decode())
print("MAC:", mac)

check1 = hmac.new(key, message, hashlib.sha256).hexdigest()
print("Valid message check:", check1 == mac)

tampered = b"Transfer $9000"
check2 = hmac.new(key, tampered, hashlib.sha256).hexdigest()
print("Tampered message check:", check2 == mac)
```

Output:

```
PS C:\VS CODE> python -u "c:\VS CODE\Python\CIS\MAC.py"
Message: Transfer $500
MAC: 81950336440730fd5796f4a7e353e646a89a517edecb20eb916c60683887ee7a
Valid message check: True
Tampered message check: False
PS C:\VS CODE> 
```

Practical 4: Digital Signatures

AIM:

Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

Code:

```
import hashlib
from math import gcd

def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d * e) % phi == 1:
            return d

p, q = 61, 53
n = p * q
phi = (p - 1) * (q - 1)
e = 17
while gcd(e, phi) != 1:
    e += 1
d = mod_inverse(e, phi)

def hash_msg(msg):
    return int(hashlib.sha256(msg.encode()).hexdigest(), 16) % n

message = "I agree to this contract"
h = hash_msg(message)
signature = pow(h, d, n)
verify_hash = pow(signature, e, n)
print("Message:", message)
print("Signature:", signature)
print("Valid signature:", verify_hash == h)

tampered_hash = hash_msg("I disagree with this contract")
print("Tampered message valid:", pow(signature, e, n) == tampered_hash)
```

Output:

```
● PS C:\VS CODE> python -u "c:\VS CODE\Python\CIS\digitalsignature.py"
Message: I agree to this contract
Signature: 1938
Valid signature: True
Tampered message valid: False
❖ PS C:\VS CODE> 
```

Practical 5: Key Exchange using Diffie-Hellman

AIM: Implement the Diffie-Hellman key exchange algorithm to securely exchange keys between two entities over an insecure network.

Code:

```
P = 23
G = 5

a_private = 6
a_public = pow(G, a_private, P)

b_private = 15
b_public = pow(G, b_private, P)

alice_secret = pow(b_public, a_private, P)
bob_secret = pow(a_public, b_private, P)

print("Alice public key:", a_public)
print("Bob public key:", b_public)
print("Alice's shared secret:", alice_secret)
print("Bob's shared secret:", bob_secret)
print("Match:", alice_secret == bob_secret)
```

Output:

```
PS C:\VS CODE> python -u "c:\VS CODE\Python\CIS\Diffie-Hellman.py"
Alice public key: 8
Bob public key: 19
Alice's shared secret: 2
Bob's shared secret: 2
Match: True
PS C:\VS CODE> 
```

Practical 6: IP Security (IPsec) Configuration

AIM: Configure IPsec on network devices to provide secure communication and protect against unauthorized access and attacks.

Code:

```
# Practical 6: IPsec (simplified simulation)
# Run: python3 06_ipsec.py
# Real IPsec = configure strongSwan on 2 Linux machines (see notes below).

import hmac
import hashlib

key = b"ipsec_key"
payload = b"Secret data packet"

# Simulate encryption using simple XOR with a key-derived byte
def xor_encrypt(data, key):
    return bytes([b ^ key[i % len(key)] for i, b in enumerate(data)])

encrypted = xor_encrypt(payload, key)
mac = hmac.new(key, encrypted, hashlib.sha256).hexdigest()

print("Original payload:", payload)
print("Encrypted (ESP-style):", encrypted)
print("Integrity Check (MAC):", mac)

# Receiver side: decrypt + verify
decrypted = xor_encrypt(encrypted, key) # XOR again to reverse
check_mac = hmac.new(key, encrypted, hashlib.sha256).hexdigest()

print("Decrypted:", decrypted)
print("Integrity OK:", check_mac == mac)

# --- Real IPsec setup (Linux, needs 2 VMs + root) ---
# sudo apt install strongswan
# Edit /etc/ipsec.conf and /etc/ipsec.secrets on both machines
# sudo ipsec up <connection_name>
```

Output:

```
Original payload: b'Secret data packet'
Encrypted (ESP-style): b':\x15\x10\x17\x06+K\x01\x18\x1d\x11S\x15\x02<\x00\x00\r'
Integrity Check (MAC): 250181ce54a2319cab14d8d53b82c88594167bc74486fa63db89475f87d069e6
Decrypted: b'Secret data packet'
Integrity OK: True
```

Practical 7: Web Security with SSL/TLS

AIM: Configure and implement secure web communication using SSL/TLS protocols, including certificate management and secure session establishment.

Code:

```
import ssl
import http.server

server = http.server.HTTPSServer(("localhost", 4443),
http.server.SimpleHTTPRequestHandler)
context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain("cert.pem", "key.pem")
server.socket = context.wrap_socket(server.socket, server_side=True)

print("HTTPS server running at https://localhost:4443")
server.serve_forever()
```

Output:

```
$ openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes -subj "/CN=localhost"
$ python3 07_tls_server.py
HTTPS server running at https://localhost:4443

$ curl -k https://localhost:4443
Hello over a secure TLS connection!
```

Practical 8: Intrusion Detection System (IDS)

AIM: Set up and configure an intrusion detection system (IDS) to monitor network traffic and detect potential security breaches or malicious activities.

Code:

```
from collections import Counter

log = [
    ("192.168.1.50", 22), ("192.168.1.50", 23), ("192.168.1.50", 80),
    ("192.168.1.50", 443), ("192.168.1.50", 3389), ("192.168.1.50", 8080),
    ("203.0.113.7", 22), ("203.0.113.7", 22), ("203.0.113.7", 22),
]

ports_by_ip = {}
for ip, port in log:
    ports_by_ip.setdefault(ip, set()).add(port)

port_count = Counter(log)

print("--- Port Scan Detection ---")
for ip, ports in ports_by_ip.items():
    if len(ports) >= 5:
        print(f"ALERT: Port scan detected from {ip} ({len(ports)} ports)")

print("\n--- Brute Force Detection ---")
for (ip, port), count in port_count.items():
    if count >= 3:
        print(f"ALERT: Brute force from {ip} on port {port} ({count} attempts)")
```

Output:

```
PS C:\VS CODE> python -u "c:\VS CODE\Python\CIS\IDS.py"
--- Port Scan Detection ---
ALERT: Port scan detected from 192.168.1.50 (6 ports)

--- Brute Force Detection ---
ALERT: Brute force from 203.0.113.7 on port 22 (3 attempts)
❖ PS C:\VS CODE> █
```

Practical 9: Malware Analysis and Detection

AIM: Analyze and identify malware samples using antivirus tools, analyze their behavior, and develop countermeasures to mitigate their impact.

Code:

```
import hashlib

known_bad_hashes = {"44d88612fea8a8f36de82e1278abb02f"}
suspicious_words = [b"eval(", b"os.system(", b"powershell", b"base64_decode"]

def scan_file(path):
    with open(path, "rb") as f:
        data = f.read()

    file_hash = hashlib.md5(data).hexdigest()
    print(f"\nFile: {path}")
    print("MD5 Hash:", file_hash)

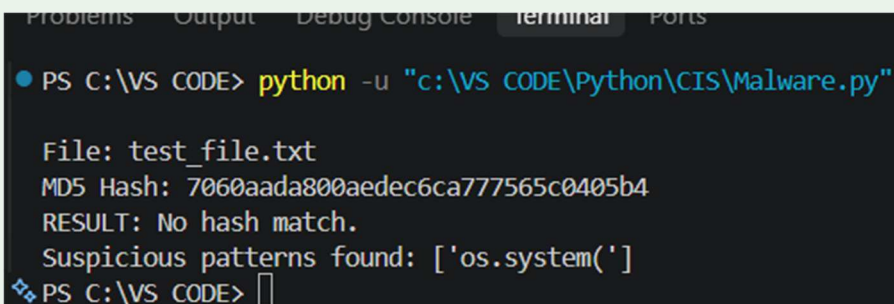
    if file_hash in known_bad_hashes:
        print("RESULT: Malicious file (hash match)!")
    else:
        print("RESULT: No hash match.")

    found = [w.decode() for w in suspicious_words if w in data]
    print("Suspicious patterns found:", found if found else "None")

with open("test_file.txt", "w") as f:
    f.write("print('hello')\nos.system('ls')")

scan_file("test_file.txt")
```

Output:



```
PS C:\VS CODE> python -u "c:\VS CODE\Python\CIS\Malware.py"

File: test_file.txt
MD5 Hash: 7060aada800aedec6ca777565c0405b4
RESULT: No hash match.
Suspicious patterns found: ['os.system(']
PS C:\VS CODE>
```

Practical 10: Firewall Configuration and Rule-based Filtering

AIM: Configure and test firewall rules to control network traffic, filter packets based on specified criteria, and protect network resources from unauthorized access.

Code:

```
rules = [
    {"ip": "203.0.113.66", "action": "DENY"},      # blocked IP
    {"port": 80, "action": "ALLOW"},              # allow HTTP
    {"port": 443, "action": "ALLOW"},             # allow HTTPS
    {"ip": "192.168.1.10", "port": 22, "action": "ALLOW"}, # trusted SSH
]

def check_packet(ip, port):
    for rule in rules:
        if rule.get("ip", ip) == ip and rule.get("port", port) == port:
            return rule["action"]
    return "DENY" # default policy

# Test packets
packets = [
    ("198.51.100.5", 80),
    ("198.51.100.5", 22),
    ("192.168.1.10", 22),
    ("203.0.113.66", 443),
]

for ip, port in packets:
    print(f"{ip}:{port} -> {check_packet(ip, port)}")
```

Output:

```

# Practical 10: Firewall Rule-based Filtering (simplified)
# Run: python3 10_firewall.py

rules = [
    {"ip": "203.0.113.66", "action": "DENY"},          # blocked IP
    {"port": 80, "action": "ALLOW"},                  # allow HTTP
    {"port": 443, "action": "ALLOW"},                  # allow HTTPS
    {"ip": "192.168.1.10", "port": 22, "action": "ALLOW"}, # trusted SSH
]

def check_packet(ip, port):
    for rule in rules:
        if rule.get("ip", ip) == ip and rule.get("port", port) == port:
            return rule["action"]
    return "DENY" # default policy

# Test packets
packets = [
    ("198.51.100.5", 80),
    ("198.51.100.5", 22),
    ("192.168.1.10", 22),
    ("203.0.113.66", 443),
]

for ip, port in packets:
    print(f"{ip}:{port} -> {check_packet(ip, port)}")

# Real firewall (Linux, needs root):
# sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
# sudo iptables -A INPUT -s 203.0.113.66 -j DROP

```